

GIJO AS 배포 가이드 (온프레미스·폐쇄망) —

v2.3.0

대상: GIJO AS를 사내 폐쇄망에 설치·운영하는 담당자(인프라/보안).

원칙: 데이터·AI가 조직 경계 밖으로 나가지 않습니다. 외부 연결은 선택적 CTI 벤더 API와 최초 모델 다운로드(망 분리 전) 뿐입니다.

1. 배포 모드 두 가지

모드	구성	언제
분산 모드 (권장·표준)	서버 1대(RTX 3090 GPU 머신)가 상주 + 데스크톱 N대가 접속	실제 운영. 여러 보안담당자가 한 서버를 공유
단독 모드 (standalone)	데스크톱 1대에 서버까지 번들(설치본에 포함)	소규모·데모·오프라인 시연

- 분산 모드: 데스크톱 앱에 `GIJO_SERVER_URL` 을 지정하면 번들 서버를 띄우지 않고 원격 서버에 붙습니다.
- 단독 모드: `GIJO_SERVER_URL` 미지정 시, 설치본에 포함된 **server-dist**를 앱이 자동 기동합니다 (Electron 내장 Node ABI로 재빌드된 상태).

2. 사전 준비 — GPU 서버 (망 분리 *전*에)

폐쇄망은 인터넷이 없으므로 **망 분리 이전에** 아래를 GPU 머신에 준비합니다.

2.1 하드웨어·OS

- GPU: **RTX 3090 (24GB)** 급 이상 권장. Windows 11 / Windows Server.
- 디스크: 모델·데이터 여유 있게(수십~수백 GB).

2.2 런타임·툴체인

- Node.js LTS** (서버 실행).
- Python 3.12** + 학습 스택(자가학습 사용 시): `unsloth` , `torch(CUDA)` , `peft` , `gguf` , `transformers` .
- llama.cpp** 빌드(로컬 LLM 서빙·GGUF 변환·양자화):

`convert_hf_to_gguf.py` + `build/bin/Release/llama-server.exe` + `llama-quantize.exe` .

- 문서 추출용: `pypdf` 등.

2.3 모델 사전 다운로드 (⚠️ 망 분리 전 필수)

폐쇄망 이전 후에는 받을 수 없으니 **미리** 내려받아 `server/models/` (또는 `GIJO_MODELS_DIR`)에 배치:

용도	예시 모델	비고
보안 채팅	Lily-Cybersecurity-7B 등 GGUF	에이전트 기본
RAG 임베딩	BGE-M3 GGUF (<code>bge-m3/bge-m3.gguf</code>)	장기 기억 필수
경량/코딩	Qwen2.5 계열 GGUF	용도별
자가학습 베이스	NousResearch/Hermes-3-Llama-3.1-8B (fp16, ~16GB)	헤르메스 학습 루프용. <code>hf download</code> 로 HF 캐시에 선다운로드

추천 LLM 가이드 화면에서 용도별 모델을 다운로드할 수 있습니다(온라인 상태일 때).

⚖️ 모델 라이선스 · BYOM (상업 배포 시 필수 확인)

번들 모델을 그대로 재배포·판매하려면 각 모델의 원본 라이선스를 지켜야 합니다. `GET`

`/api/localengine/model-licenses` (관리자)로 현재 모델들의 분류를 확인하세요:

- **permissive** (예: Qwen2.5-7B/Coder-Qwen3-BGE-M3, Apache-2.0 추정) — 상업 번들 가능(추정, 원본 확인 권장)
- **restricted** (Llama-Hermes = Llama 커뮤니티 라이선스, Qwen2.5-3B = 비상업) — 조건 준수/제외 필요
- **byom** (합성·개조·출처불명: gijo 오케스트레이터·merged·abliterated 등) — **번들 금지, 고객이 직접 (BYOM)**
- **상용 배포 권장**: 기본 탑재는 permissive만, 나머지는 고객이 사내에서 직접 받아 `GIJO_MODELS_DIR`에 배치.

⚠️ 이 분류는 법무 검토의 출발점이며 법적 확정이 아닙니다. 판매 전 각 모델 원본 라이선스를 법무가 확인하세요.

3. 서버 설치·기동 (GPU 머신)

3.1 배치

1. `server/` 디렉터리를 GPU 머신에 복사.
2. 의존성 설치: `npm ci --omit=dev` (`server/`).
3. `server/models/` 에 사전 다운로드한 GGUF 모델 배치.
4. `server/llama.cpp/` 에 빌드된 바이너리 배치.

3.2 주요 환경변수

변수	기본값	설명
GIJO_SERVER_PORT	4000	서버 포트
GIJO_DB_PATH	data/gijo-as.sqlite	SQLite 파일 경로
GIJO_MODELS_DIR	models	모델 디렉터리
GIJO_LLAMA_CPP_DIR	llama.cpp	llama.cpp 경로
GIJO_EMBEDDING_MODEL_ID	bge-m3	RAG 임베딩 모델
PYTHONUTF8	—	1 고정 권장 (Windows cp949로 한글 로그 깨짐·스크립트 사망 방지)
보안 (self-install 필수)		
NODE_ENV	—	운영 설치는 production 권장(안전 기본값·시크릿 강제)
GIJO_JWT_SECRET	(개발 기본값)	운영 필수 — 미설정 시 production 에서 서버가 뜨지 않음(토큰 위조 방지). 긴 랜덤 문자열
GIJO_INITIAL_ADMIN_PASSWORD	—	최초 관리자 비번. 미지정+ production 이면 랜덤 생성 후 콘솔 1회 출력
GIJO_INITIAL_ADMIN_USERNAME	admin / jyh	최초 관리자 아이디
GIJO_TLS_CERT_PATH / GIJO_TLS_KEY_PATH	—	둘 다 지정 시 HTTPS 로 서빙(사내망 스니핑 방지 권장)
GIJO_MIN_PASSWORD_LEN	8	비밀번호 최소 길이
GIJO_LOGIN_MAX_FAILS	10	로그인 실패 임계(초과 시 15분 잠금)

3.3 기동

```
# server/ 에서
set PYTHONUTF8=1
node dist/index.js
```

- 기동 로그: `GIJO AS 서버 기동 - http://localhost:4000 (WebSocket: /ws)`
- 로컬 엔진(llama-server)이 모델을 로드합니다(모델이 배치돼 있어야 함).

3.4 상시 상주

- 서버는 항상 켜져 있어야 합니다. **Windows 서비스**(nssm 등) 또는 작업 스케줄러로 부팅 시 자동 기동 등록을 권장합니다.

- **방화벽:** 사내망에서 데스크톱들이 불도록 4000/tcp (및 WebSocket) 허용. 외부(인터넷) 노출은 금지.

3.5 최초 관리자 계정 (⚠ self-install 보안 핵심)

운영 설치에 **알려진 기본 비밀번호를 절대 쓰지 않도록** 아래 중 하나로 초기 계정을 만드세요.

- **권장:** `GIJO_INITIAL_ADMIN_PASSWORD` (+선택 `GIJO_INITIAL_ADMIN_USERNAME`)를 지정하고 기동 → 그 값으로 관리자 생성.
- 또는 `NODE_ENV=production` 만 설정하고 초기 비번 미지정 → **강력 랜덤 비번을 생성해 콘솔에 1회만 출력합니다**(설치자가 기록·로그인·즉시 변경).
- (개발/데모 한정) 위 둘 다 없으면 `jyh` / `changeme` 가 시드됩니다 — **운영에서는 금지**.
- 최초 로그인 후 담당자별 계정을 발급하세요(설정 → 계정 관리). 로그인 실패 `GIJO_LOGIN_MAX_FAILS` 회 초과 시 15분 잠깁니다(무차별 대입 방어).

3.6 설치 후 자가 진단 (프리플라이트)

관리자로 로그인 후 `GET /api/admin/preflight` 를 호출하면 이 머신의 준비 상태를 항목별로 점검합니다: Node-GPU(`nvidia-smi`)·llama-server·모델(상업·BYOM 개수)·JWT 시크릿·**기본 비밀번호 사용 여부**·데이터 저장소. `ready:false` 면 `fail` 항목을 조치하세요.

- `/api/health` 는 스키마 버전을 함께 반환합니다(업그레이드 후 반영 확인용).

3.7 (대안) WSL2 서버 배포

Windows에 직접 설치하는 대신 같은 GPU 머신의 **WSL2(Ubuntu 24.04)** 안에서 서버를 돌리는 방식입니다. UTF-8 기본 로케일이라 `PYTHONUTF8` 우회가 불필요하고, CUDA 생태계의 1차 지원 대상(Linux)을 그대로 씁니다.

클라이언트 배포(\$4)는 동일합니다. 배경·전체 절차는 `GIJO_AS_WSL2_서버이전_계획서.md` 참조.

3.7.1 WSL 환경

```
# systemd 활성화 - /etc/wsl.conf 에 아래를 넣고 `wsl --shutdown` 후 재진입
[boot]
systemd=true

sudo apt install -y build-essential cmake python3.12-venv
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash - && sudo apt install -y nodejs

# CUDA toolkit (호스트 드라이버는 건드리지 않는다 - WSL 안에는 toolkit만)
wget https://developer.download.nvidia.com/compute/cuda/repos/wsl-ubuntu/x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb && sudo apt update
sudo apt install -y cuda-toolkit-13-3
nvidia-smi # GPU 인식 확인
```

3.7.2 llama.cpp 빌드

```
git clone --depth 1 https://github.com/ggml-org/llama.cpp.git ~/gijo-as/llama.cpp
cd ~/gijo-as/llama.cpp
export PATH=/usr/local/cuda/bin:$PATH
cmake -B build -DGGML_CUDA=ON -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=86
cmake --build build --config Release -j$(nproc)
build/bin/llama-server --list-devices # CUDA0 가 보여야 한다
```

CMAKE_CUDA_ARCHITECTURES 는 GPU에 맞춘다(RTX 3090 = 86, RTX 40xx = 89).

3.7.3 배치

~/gijo-as/ 아래 server/ (소스·scripts·package*.json·docs·manifest.json), venv/ , docs/ , llama.cpp/ 를 둡니다.

모델과 DB는 반드시 WSL ext4 내부에 둡니다 — /mnt/d/... 에 두면 9p 파일시스템 경계 때문에 모델 로드·SQLite I/O가 수 배 느려집니다.

```
cd ~/gijo-as/server
npm ci && npx tsc -p tsconfig.json # better-sqlite3·lancedb가 Linux용으로 재빌드된다
python3 -m venv ~/gijo-as/venv && ~/gijo-as/venv/bin/pip install -r requirements.txt
```

DB는 파일 복사가 아니라 **온라인 백업 API**로 옮깁니다(가동 중이면 -wal 에 미반영 트랜잭션이 남아 단순 cp 는 깨질 수 있음):

```
node -e 'const D=require("better-sqlite3");const s=new D("/mnt/d/Connect AI/server/data/gijo-as.sqlite",{readonly:true});s.backup("data/gijo-as.sqlite").then(()=>s.close())'
```

data/ 아래 encryption.key · memory.lancedb/ · kev.json 도 함께 옮깁니다.

docs-manifest.json 이 열거한 **고객사용 제품 문서**는 GIJO_DOCS_DIR 가 가리키는 디렉터리에 두어야 부팅 시 RAG 인입이 됩니다.

3.7.4 systemd 상주

/etc/systemd/system/gijo-as.service — venv를 PATH 앞에 두어 spawn("python") 이 venv를 잡게 하는 것이 핵심입니다.

```
[Service]
Type=simple
User=gijo
WorkingDirectory=/home/gijo/gijo-as/server
EnvironmentFile=/home/gijo/gijo-as/gijo-as.env
Environment=PATH=/home/gijo/gijo-
as/venv/bin:/usr/local/cuda/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
Environment=LD_LIBRARY_PATH=/usr/local/cuda/lib64:/usr/lib/wsl/lib
ExecStart=/usr/bin/node dist/index.js
Restart=always
```

gijo-as.env (권한 600, 커밋 금지)에는 `NODE_ENV=production`, `GIJO_JWT_SECRET` (필수 — 운영에서 미설정이면 서버가 뜨지 않음), `GIJO_SERVER_PORT=4000`, 그리고 `llama.cpp`를 `server/` 밖에 두었으므로 `GIJO_LLAMA_SERVER_PATH` · `GIJO_LLAMA_CPP_DIR` · `GIJO_DOCS_DIR` 절대경로를 넣습니다.

```
sudo systemctl daemon-reload && sudo systemctl enable --now gijo-as.service
```

3.7.5 코드 갱신 배포 (2회차 이후)

최초 배치(3.7.3) 이후의 코드 반영은 `server/scripts/deploy.sh` 한 줄로 끝냅니다.

유닛이 `User=gijo` + `Restart=always` 라 **sudo 없이** 전 과정이 자동화됩니다.

```
# Windows에서 (PowerShell - Git Bash는 /mnt 경로를 Windows 경로로 망가뜨립니다)
wsl -d Ubuntu-24.04 -- bash "/mnt/d/Connect AI/server/scripts/deploy.sh" --dry-run # 반영될 파일 확인
wsl -d Ubuntu-24.04 -- bash "/mnt/d/Connect AI/server/scripts/deploy.sh" # 실제 배포
```

스크립트가 하는 일 — ① 변경된 `.ts` 탐지 ② 복사 ③ `tsc` ④ `pkill` ⑤ health 폴링:

- **변경 탐지는 `diff --strip-trailing-cr` 로 합니다.** Windows 체크아웃은 CRLF, WSL은 LF라 그냥 비교하면 모든 파일이 바뀐 것으로 나와 실제 변경분을 분간할 수 없습니다. 복사할 때도 `tr -d '\r'` 로 걷어냅니다.
- **`src/` 만 배포합니다.** 운영 서버에는 `test/` 가 없습니다(런타임 불필요). 테스트까지 동기화하면 배포마다 테스트 수십 개가 운영에 깔립니다.
- **`tsc` 가 실패하면 재시작하지 않습니다.** 깨진 `dist` 로 서비스를 내리는 것이 최악입니다.
- 재기동은 systemd가 맡습니다(`Restart=always`). 스크립트는 health 200을 확인할 때까지 최대 30초 기다립니다.

⚠ **재시작하면 접속 중인 세션이 전부 끊깁니다.** refresh 토큰이 인메모리라 전 사용자 재로그인이 필요합니다. 근무 시간대에는 접속자를 확인하고 실행하세요.
서버 코드만 바뀐 배포는 클라이언트 설치본 재빌드가 **불필요**합니다(\$4는 렌더러·Electron 변경 시에만).

3.7.6 네트워킹 (분산 모드)

- **권장(Windows 11 22H2+):** %USERPROFILE%\wslconfig 에 [wsl2] / networkingMode=mirrored → WSL이 호스트 IP를 공유해 포트포워딩이 불필요합니다. 적용에 `wsl --shutdown` 이 필요하며, **이때 Windows 쪽 기존 서버가 4000을 잡고 있으면 충돌하므로** 전환 시점에 함께 정리합니다.
- **구버전 대안:** `netsh interface portproxy add v4tov4 listenport=4000 connectaddress=<WSL IP> connectport=4000` (WSL IP는 재부팅 시 바뀌므로 갱신 스크립트 필요).
- Windows 방화벽 `4000/tcp` 인바운드 허용(사내망 한정).
- **부팅 시 자동 기동:** 작업 스케줄러에 시작 트리거로 `wsl.exe -d Ubuntu-24.04 --exec /bin/true` 를 SYSTEM·최고 권한으로 등록하면 WSL이 올라오면서 systemd가 서비스를 이어서 띄웁니다. **재부팅 테스트를 전환 전 필수 통과 조건으로** 삼으세요.

3.7.7 롤백

Windows 쪽 서버·모델·DB를 지우지 않고 그대로 둡니다. 문제 시 `sudo systemctl stop gijo-as` 후 기존 Windows 서비스를 재기동하면 즉시 복귀합니다.

단, 전환 후 WSL DB에 변경분이 쌓이므로 **롤백 시에는 WSL DB를 Windows로 역복사**해야 합니다.

4. 클라이언트(데스크톱) 배포

4.1 설치본

- 빌드 산출물: `client/release/GIJO AS Setup 2.3.0.exe` (NSIS 설치본, 약 158MB).
- 빌드 방법(개발 PC): `client/` 에서 `npm run dist`

→ 내부적으로 `build` (TS+preload) → `build-server-dist` (server 사본 + better-sqlite3를 Electron ABI로 재 빌드) → `electron-builder` (nsis) 순으로 실행됩니다.

- 재빌드 후 정리: 빌드가 `client/server-dist/package.json` 의 버전 문자열을 바꾸므로(추적 산출물), `git checkout -- client/server-dist/package.json` 로 되돌려 트리를 clean하게 유지합니다. `release/` 는 gitignore입니다.

4.2 배포·설치

1. 설치본을 각 보안담당자 PC에 배포 → 실행해 설치(사용자 단위 설치, 관리자 권한 불필요).
2. 최초 실행 시 **서버 주소**를 입력(분산 모드): `http://<서버IP>:4000` .
3. 또는 실행 환경변수 `GIJO_SERVER_URL=http://<서버IP>:4000` 로 지정하면 번들 서버를 띄우지 않고 원격 서버에 붙습니다.
4. 관리자에게 받은 **아이디/비밀번호**로 로그인.
5. 업그레이드: 새 버전 설치본을 실행하면 덮어쓰기 설치됩니다. 로컬 설정(로그인 서버 주소·탐색기 루트 등 `%APPDATA%/GIJO AS`)은 유지됩니다.

단독 모드로 쓰려면: `GIJO_SERVER_URL` 을 지정하지 않고 실행하면 설치본에 포함된 서버가 자동 기동됩니다(모델은 별도 배치 필요).

4.3 v2.3.0 클라이언트 신규 화면·기능 (담당자 안내용)

설치 후 담당자에게 아래 변경점을 안내하세요. 자세한 사용법은 `GIJO_AS_보안담당자_실무매뉴얼.md`.

-  **MCP 연동**(신규 화면, 관제·모니터링 그룹): AI 에이전트를 외부 도구·데이터(위협 인텔·스캐너·티켓·SIEM)에 MCP로 연결. **기본 OFF·관리자 게이팅**, 외부로 데이터가 나가는 **원격(HTTP) 서버는 egress 승인**해야 활성화, 에이전트×도구 허용·호출 감사. *실제 연결(도구 호출)은 백엔드 MCP 클라이언트 연동 시 활성화 — 현재는 서버 정의·정책 관리 단계.*
- **전 페이지 공용 지휘 콘솔**: 대시보드 외 모든 화면 오른쪽 가장자리의  **작업 화면** 탭 → 어느 화면에서든 AI에게 자연어로 지시(작업 세션+지휘 콘솔). 화면 경로가 맥락으로 함께 전달됩니다.
- **내 PC 파일 탐색기**: 대시보드 왼쪽이 담당자 **내 문서** 기준 파일 탐색기로. 문서 파일 클릭 → 장기 기억(RAG)에 바로 올리기. 아래 **AS 탐색기**에 등록 리포트·문서 목록.
- **AI 팀 · 전체메뉴 · 툴바**: 대시보드에  **AI 팀 울타리**(에이전트+엔진+가드레일,  팀 사무실 열기), 오른쪽 상단  **전체메뉴** (→ 보안 KPI), 상단 툴바는 **내 업무 바로가기 등록 항목만** 표시.
- **보안 KPI 접이식**: 섹션 제목 클릭으로 접기/펼치기(기본 접힘, 추세 번다운 2섹션만 상단·펼침).
- **폰트 통일**: 전 화면 Pretendard(오프라인 번들, 폐쇄망에서도 CDN 없이 동작).

5. 폐쇄망 반입 체크리스트

- [] Node.js / Python 3.12 / (학습 시) unsloth·torch(CUDA)·peft·gguf·transformers
- [] llama.cpp 빌드 산출물(server·quantize·convert 스크립트)
- [] GGUF 모델: 보안 채팅 · **BGE-M3(임베딩)** · 경량
- [] (자가학습 시) **Hermes-3-8B fp16 ~16GB** HF 캐시
- [] `server/` 코드 + `npm ci --omit=dev` 완료된 node_modules
- [] `client/release/GIJO AS Setup 2.3.0.exe`
- [] 서버 상시 상주 등록(서비스) · 방화벽 `4000/tcp`
- [] **보안**: `GIJO_JWT_SECRET` 설정 · 초기 관리자 비번(env 또는 랜덤) · (권장) TLS 인증서
- [] **설치 후**: `GET /api/admin/preflight` 로 `ready:true` 확인 · 기본 비밀번호 변경
- [] **모델**: 라이선스 확인(`/api/localengine/model-licenses`) — 번들 가능/BYOM 구분

6. 업그레이드·재배포

1. 서버: 새 `server/dist` 반영 후 **서버 프로세스 재시작**(무중단이 아니므로 점검 시간에).
2. 클라이언트: 새 버전 설치본 재배포(버전 번호로 구분).
3. DB(`data/gijo-as.sqlite`)는 유지됩니다 — 스키마 변경은 기동 시 자동 마이그레이션(ALTER 안전 처리 + `schema_migrations` 기록). 업그레이드 후 `GET /api/health` 의 `schema` 버전으로 반영을 확인하세요.
4. **업그레이드 전 백업**: 서버 정지 후 `data/gijo-as.sqlite` 파일을 복사(권장). 문제 시 파일을 되돌리면 복구됩니다.

주의: 서버 코드만 갱신하고 재시작하지 않으면 **구버전 프로세스가 계속 떠 있어** 새 기능/라우트가 반영되지 않습니다(구버전은 새 API를 404로 응답). 갱신 후 반드시 재시작하세요.

7. 문제 해결

- 데스크톱이 서버에 못 붙음: 서버 주소/포트 확인 → 방화벽 `4000/tcp` → 서버 상주 여부.
- 한글 로그 깨짐 / python 스크립트 사망: `PYTHONUTF8=1` 설정 확인.
- 채팅/RAG가 일시 불가: 학습 루프·파인튜닝 중에는 GPU 확보를 위해 추론 엔진이 잠시 정지됩니다(완료 후 자동 재기동).
- 임베딩(RAG) 미동작: `GIJO_MODELS_DIR` 에 `bge-m3/bge-m3.gguf` 배치 여부 확인.
- 자가학습 실행 안 됨: 학습 루프 화면의 사전 점검(preflight) 에서 python·학습 스택·베이스 모델 캐시 상태 확인.
- 단독 모드에서 SQLite 오류: 번들 서버의 better-sqlite3가 Electron ABI로 재빌드됐는지(설치본 빌드 시 `build-server-dist` 가 수행) 확인.
- 설치했는데 뭐가 안 되는지 모를 때: `GET /api/admin/preflight` 로 항목별 진단(GPU·모델·JWT·기본 계정·저장소). `fail` 먼저 해결.
- 운영인데 서버가 안 뜸: `NODE_ENV=production` 인데 `GIJO_JWT_SECRET` 미설정이면 의도적으로 기동을 막습니다(취약 상태 방지). 시크릿을 설정하세요.
- 초기 관리자 비번을 못 봤음: 랜덤 생성 비번은 콘솔에 1회만 출력됩니다. 놓쳤으면 서버 로그 (`data/restart-out.log` 등)를 확인하거나, DB의 users를 비우고(백업 후) 재기동해 재생성.

*본 가이드는 GIJO AS **v2.3.0**(2026-07-20) 기준입니다. 서버 보안(안전 초기계정·JWT·TLS·프리플라이트·모델 라이선스·BYOM)은 §2·§3·§5, 클라이언트 배포·신규 화면은 §4를 참고하세요. 사용법은 `GIJO_AS_보안담당자_실무매뉴얼.md`.*